Change Request revisionata: integrazione Docker Compose in docker-dev

- Revisione: 2 — consolidamento della verifica di applicabilità
- Data: 13 ottobre 2026
- Stato: proposta tecnica; decisioni aperte da approvare prima dell'implementazione
- Sostituisce come riferimento progettuale la CR precedente; non costituisce codice implementato o collaudato.

1. Obiettivo e perimetro

Integrare Docker Compose per affiancare servizi opzionali al container di sviluppo, preservando il funzionamento legacy. Usare Perl, YAML::XS e Template::Toolkit per preparare la successiva migrazione dell'orchestratore Bash a Perl.

Requisiti confermati:

- `langs/<tipo>/` descrive l'ambiente di sviluppo.
- `services/<tipo>/` descrive ogni servizio riutilizzabile.
- Il resolver è generico: nessuna logica PostgreSQL, Redis o altro servizio nel programma centrale.
- I template Perl hanno suffisso `compose.yaml.tt`.
- Il container di sviluppo è effimero per default; persistenza esplicita nei volumi.
- Il progetto può fornire personalizzazioni Compose che non vengono sovrascritte dal generatore.

Non sono inclusi automaticamente: migrazione completa del main a Perl, traduzione generale di opzioni docker run, migrazione dei dati database, cambio della politica di sincronizzazione uv, riparazione automatica dei permessi.

2. Baseline esaminata e limiti della verifica

File di riferimento:

- `docker-dev/docker-dev.sh`
- `docker-dev/apply-templates.sh`
- `docker-dev/docker-run-extra-args.sh`
- `docker-dev/langs/Python/templates/Dockerfile`
- `docker-dev/langs/Python/templates/docker-run-extra-args`
- `docker-dev/langs/Python/templates/entrypoint.Python.sh`

- `docker-dev/langs/Python/bin/10.setup-volumes.sh`
- `docker-dev/langs/Python/bin/20.build.sh`
- `docker-dev/langs/Python/bin/30.run.sh`
- CR precedente `change-request-docker-compose-docker-dev.md`.

La revisione è statica/documentale. Il controllo `bash -n` sui sette script shell nella precedente verifica è passato; non sono stati eseguiti build o avvii di container.

La baseline usa discovery dei file numerati e ordinamento `sort -z -V`, poi esecuzione sequenziale come processi figli. L'interfaccia dei figli è:

```
PROJECT_DIR IMAGE_NAME CONTAINER_NAME [command arg ...]
```

Naming esistente:

```
IMAGE_NAME     = lowercase(P_TARGET.P_TYPE)
CONTAINER_NAME = IMAGE_NAME.P_NAME
VOLUME_NAME    = venv.CONTAINER_NAME   # salvo override ambiente
```

Il caso `dev.python.foo` è un esempio, non un'identità da hardcodificare: `--penv`, `-i`, ambiente e normalizzazione del basename influenzano i valori.

Runtime Python:

```
progetto host → /app
venv.<container_name> → /app/.venv
uv-python → /home/<utente>/.local/share/uv/python
bashrc → /app/.docker_container/.bashrc, disponibile attraverso il bind mount
```

Il Dockerfile definisce USER, WORKDIR, PATH ed ENTRYPOINT. Senza comando l'entrypoint apre Bash con la bashrc del progetto; con comando esegue `exec "$@"`.

Il default del resolver extra args è `--rm -it`; il template Python consegnato al progetto contiene `--rm -it -p 8000:8000`. Override espliciti possono cambiare tale comportamento.

3. Correzioni obbligatorie della CR precedente

ID	Problema	Correzione richiesta
C01	40.services viene dopo il run foreground	Preparare i servizi prima di 30.run oppure introdurre fasi esplicite
C02	Project name Compose con punti	Identità Compose separata e validata
C03	up generale seguito da run dev	Avviare solo i servizi di supporto; dev one-off
C04	apply-templates presunto renderer TT	Renderer Perl distinto dalla copia dei file
C05	Runtime non completamente preservato	Riprodurre /app, entrypoint, utente, uv-python, bashrc e porta 8000

ID	Problema	Correzione richiesta
C06	Manifest incompleto e alias impliciti	Schema versionato, alias dichiarativi, parametri tipizzati
C07	CLI con virgole ambigue	Separare selezione servizi e opzioni
C08	Frammenti YAML concatenati	Parsing e aggregazione strutturata con conflitti rilevati
C09	Interpolazione non serializzata	Filtri per scalari/strutture e gestione interpolazione Compose
C10	Variabili figlie non garantite	Contesto esplicito e argomenti preservati
C11	Extra args legacy ignorabili	Politica esplicita, nessuna perdita silenziosa
C12	Operazioni amministrative attraverso pipeline completa	Dispatch anticipato e stato applicato persistente

4. Architettura proposta

```
docker-dev/
├── docker-dev.sh
├── apply-templates.sh
├── docker-run-extra-args.sh
├── bin/
│   └── docker-dev-services.pl
├── lib/
│   └── DockerDev/                # moduli Perl, organizzazione da finalizzare
├── langs/Python/
│   ├── templates/
│   │   ├── Dockerfile
│   │   ├── entrypoint.Python.sh
│   │   ├── docker-run-extra-args
│   │   └── .docker_container/
│   ├── compose/
│   │   └── compose.yaml.tt
│   └── bin/
│       ├── 10.setup-volumes.sh
│       ├── 20.build.sh
│       ├── 25.prepare-services.sh # opzione raccomandata, non ancora approvata
│       └── 30.run.sh
└── services/<tipo>/
    ├── service.yaml
    ├── templates/
    │   ├── compose.yaml.tt
    │   └── Dockerfile            # solo per immagini personalizzate
    └── bin/
        ├── 10.setup-volumes.sh   # opzionale
        └── 20.build.sh           # opzionale
```

Il template Compose di linguaggio è separato dai file consegnati al progetto per evitare che la copia ricorsiva lo distribuisca accidentalmente. È una proposta di layout, non un requisito già approvato.

Responsabilità:

Componente	Responsabilità
main Bash	CLI, progetto, operazione, piano, conferma e orchestrazione
apply-templates	Copia no-clobber e composizione bashrc; non esegue TT
resolver Perl	Catalogo, alias, manifest, parametri, contesto risolto e piano
renderer Perl	Rendering base/servizi, parsing frammenti, aggregazione e serializzazione
hook servizio	Build e inizializzazione specifiche del servizio
run/dispatcher	Backend legacy o Compose, senza duplicare il resolver

L'implementazione iniziale può condividere resolver e renderer in un unico programma Perl, purché le responsabilità siano distinguibili e testabili.

5. Registro delle decisioni aperte

Le raccomandazioni seguenti non rappresentano approvazioni dell'utente.

ID	Stato	Chiarimento o scelta	Opzioni	Raccomandazione
D01	DA DECIDERE	Ordine pipeline	Wrapper 25.prepare-services / fasi esplicite con 40.services	Wrapper 25 nella prima versione
D02	DA DECIDERE	Attivazione backend	--with implicito / --backend=compose esplicito, anche senza servizi	Supportare backend esplicito, --with come scorciatoia
D03	DA DECIDERE	Identità e collisioni	Slug legacy / slug con hash percorso / ID stabile progetto	ID stabile o hash; non rinominare volumi esistenti automaticamente
D04	DA DECIDERE	Nome container dev	Nome Compose one-off / --name legacy	Documentare differenza; usare --name solo con politica anti-collisione
D05	DA DECIDERE	Vita dei servizi	Restano dopo shell / arresto automatico / opzione dedicata	Restano fino a --down
D06	DA DECIDERE	Readiness	Compose up --wait / dipendenze in run / attesa orchestrata	up selettivo --wait se supportato, verificando assenza healthcheck
D07	DA DECIDERE	Versione minima Compose	Versione fissata / feature detection	Fissare versione testata e controllare feature richieste
D08	DA DECIDERE	Alias e nomi logici	Catalogo globale / override progetto / nomi espliciti di istanza	Globale + override progetto, precedenza documentata
D09	DA DECIDERE	Grammatica CLI	Opzioni separate / sintassi compatta con separatori diversi	--with ripetibile + --service-option
D10	DA DECIDERE	Precedenza configurazione	Base → servizi → utente / base → utente → servizi	Utente per ultimo, con controlli sulle invarianti
D11	DA DECIDERE	Immagini mancanti	Errore / build-pull automatico / opt-in	Azioni visibili nel piano e autorizzate esplicitamente
D12	DA DECIDERE	Volumi servizi	Compose-managed / external per tutti / per manifest	Scelta per manifest; setup specifico per external

ID	Stato	Chiarimento o scelta	Opzioni	Raccomandazione
D13	DA DECIDERE	Porte host	Pubblicazione default / solo richiesta / localhost	Preservare 8000 legacy; servizi solo se richiesti, localhost
D14	DA DECIDERE	Extra args legacy in Compose	Traduzione limitata / errore / warning	Nessuna traduzione generale; errore per personalizzazioni non gestite
D15	DA DECIDERE	Concorrenza	Vietata stesso progetto / lock / stack distinti	Lock progetto e file temporanei univoci
D16	DA DECIDERE	Stato e cleanup	Snapshot singolo / versioni stato / comando clean	Snapshot applicato e clean separato con conferma
D17	DA DECIDERE	Correzioni baseline	Stessa CR / intervento separato	Separare salvo problemi necessari all'integrazione
D18	DA DECIDERE	Segreti	env locale / file secret / provider esterno	Non versionare o registrare segreti nei lock/log
D19	DA DECIDERE	Aggiornamento virtualenv	Policy attuale / sync sempre / opzione	Preservare policy attuale, cambiamento separato
D20	DA DECIDERE	Template di progetto	Solo centrali / copia modificabile / override template	Centrali + compose.user.yaml; override template futuro

6. Identità e naming

Separare almeno:

```
IMAGE_NAME           = dev.python
CONTAINER_NAME       = dev.python.foo
COMPOSE_PROJECT_NAME = dev-python-foo  # esempio, soggetto a D03
VOLUME_NAME          = venv.dev.python.foo
```

Il project name Compose deve usare solo minuscole, cifre, trattini e underscore e iniziare con minuscola o cifra. Non utilizzare `dev.python.foo` con `-p`.

I nomi reali dei volumi legacy rimangono invariati e sono referenziati tramite chiavi logiche e `name:`.

```
Per servizi nuovi evitare collisioni tra linguaggi, ambienti, progetti omonimi e più
```

Chiarimenti richiesti: --penv diversi devono sempre isolare database? Due checkout dello stesso progetto condividono dati? L'ID resta invariato quando la directory viene spostata?

7. CLI proposta e compatibilità

```
docker-dev --target=Python/foo

docker-dev --target=Python/foo --with=db,cache
```

```
docker-dev --target=Python/foo \
  --with=db --with=cache \
  --service-option=db.tag=16 \
  --service-option=db.host_port=15432 \
  -- pytest -q

docker-dev --target=Python/foo --backend=compose --config

docker-dev --target=Python/foo --down

docker-dev --build-service=postgres,redis
```

Gli ultimi due tipi di opzioni sono proposte soggette a D02/D09. Non mischiare virgole come separatore servizi e parametri.

Il parser deve rispettare `--`: dopo il delimitatore non interpreta più opzioni dell'orchestratore. Preservare argomenti contenenti spazi o stringhe vuote; non ricostruire il comando con eval o shell string.

`--build-service` deve poter essere gestito prima dell'obbligo --target. Incompatibilità tra operazioni devono produrre errore chiaro.

8. Contesto tra main e processi figli

Non affidarsi a variabili Bash non esportate. Trasmettere un contesto JSON versionato con valori risolti:

```
{
  "schema_version": 1,
  "operation": "session",
  "backend": "compose",
  "project": {
    "directory": "/percorso/REP/DEV/Python/foo",
    "type": "Python",
    "environment": "DEV",
    "name": "foo",
    "id": "da-definire"
  },
  "runtime": {
    "image_name": "dev.python",
    "container_name": "dev.python.foo",
    "compose_project_name": "dev-python-foo",
    "app_dir": "/app"
  },
  "host": {"user": "utente", "uid": 1000, "gid": 1000},
  "volumes": {"venv": "venv.dev.python.foo", "uv_python": "uv-python"},
  "requested_services": [{"instance": "db", "type": "postgres", "options": {}}]
}
```

Lo schema è proposto. Il comando può viaggiare separatamente come argv oppure come lista JSON, senza shell interpolation. Il main rimane proprietario del contesto; figli indipendenti non comunicano cambiamenti al padre semplicemente assegnando variabili.

9. Schema manifest e contratto template

Definire `schema_version`, identità, immagine, parametri ammessi, profili, volumi, hook e integrazione dev. `container_port` e `connection_env` non sono obbligatori per ogni servizio.

Il resolver non deve contenere alias hardcoded, URL PostgreSQL, opzioni redis-server o chown specifici. Questi provengono da manifest/template/hook.

Contratto proposto:

- `image`: riferimento risolto; distinguere repository+tag da riferimenti completi o digest.
- `parameters`: nomi consentiti, tipi, default, limiti e valori obbligatori.
- `profiles`: lista coerente tra documento e argomenti Compose.
- `volumes`: lista con chiave logica, nome reale, mount e ownership mode.
- `hooks`: operazioni consentite, percorsi relativi confinati al servizio, input e output documentati.
- `dev_integration`: ambiente e dipendenze; collisioni rilevate.
- `healthcheck`: opzionale; scelta readiness esplicita.

Manifest PostgreSQL e Redis della CR iniziale restano esempi da completare. `REDIS_ARGS` non è un comportamento garantito: richiede un entrypoint che la consumi, oppure un `command` dichiarato dal template Redis. Le credenziali di esempio sono solo sviluppo locale.

Con STRICT => 1 tutte le variabili consultate devono esistere. Usare flag sempre definiti `has_healthcheck`, `has_data_volume`, `publish_port`; evitare controlli su nomi non presenti nel contesto.

10. Pipeline di generazione YAML

```
manifest → parsing → validazione → risoluzione → contesto template
         → rendering TT → parsing frammento → aggregazione strutturata
         → serializzazione unica → verifica Compose completo
```

Non concatenare frammenti con `services:` ripetuto o più documenti YAML. Generare un solo modello con mappings services/volumes/networks.

Regole richieste:

1. Rifiutare duplicate keys nei manifest e nei frammenti attraverso controlli adatti al parser scelto.
2. Rifiutare nomi istanza duplicati e dichiarazioni incompatibili di volume/rete.
3. Aggregare una sola sezione `services.dev.environment` e `depends_on`.
4. Fallire su collisioni di variabili di connessione salvo override autorizzato.
5. Usare filtri di serializzazione per scalari, liste, mapping e booleani; non interpolare direttamente arrayref Perl.

6. Preservare stringhe come password, valori con #, :, newline, virgolette e parole interpretabili come booleani.
7. Gestire separatamente `${...}` e `$` interpretati successivamente da Compose: definire quando preservare espressioni e quando rendere un valore letterale.
8. Validare la configurazione con tutti i file, profili e ambiente effettivi, non il solo file servizi.
9. Scrivere temporanei e pubblicare atomicamente solo risultati validi; evitare output parziale.

Il merge interno dei frammenti non deve tentare di emulare tutta la semantica Compose. Aggrega contributi controllati; gli override esterni sono elaborati da Compose con precedenza definita in D10.

11. Compose base Python

Scheletro illustrativo; nomi e percorsi vengono risolti dal renderer:

```
services:
  dev:
    image: dev.python
    hostname: dev.python.foo
    stdin_open: true
    tty: true
    working_dir: /app
    volumes:
      - type: bind
        source: /percorso/assoluto/progetto
        target: /app
      - type: volume
        source: python-venv
        target: /app/.venv
      - type: volume
        source: uv-python
        target: /home/utente/.local/share/uv/python
    ports:
      - "8000:8000"
volumes:
  python-venv:
    external: true
    name: venv.dev.python.foo
  uv-python:
    external: true
    name: uv-python
```

Non sovrascrivere ENTRYPOINT, USER, PATH o comando con sleep infinity. Senza comando il normale entrypoint deve aprire la shell con bashrc. Evitare mount che nascondano l'intera home.

Tutti i percorsi generati devono essere assoluti oppure usare una base esplicitamente definita; non assumere che ogni frammento abbia la propria directory di risoluzione.

La pubblicazione 8000 replica il template attuale; il passaggio a loopback è una scelta esplicita, non una compatibilità identica.

12. Esecuzione e readiness

Non eseguire `up -d` dell'intero stack seguito da `run dev`.

Piano raccomandato:

```
# compose_args include project name valido, file esistenti e profili selezionati
# selected_services è un array di nomi logici validati
docker compose "${compose_args[@]}" config --quiet

docker compose "${compose_args[@]}" up -d "${selected_services[@]}"

# La readiness va garantita con la strategia approvata in D06.
docker compose "${compose_args[@]}" \
  run --rm --service-ports dev "${CONTAINER_CMD[@]}"
```

Questo è uno schema, non codice pronto: se l'array servizi è vuoto, saltare up; altrimenti up senza nomi avvierebbe anche dev. Valutare `--wait` sulla versione minima scelta. Se readiness è gestita esternamente si può valutare `run --no-deps`, ma solo dopo aver garantito le dipendenze.

`--service-ports` è necessario per pubblicare le porte del servizio dev in run. Il nome one-off non coincide automaticamente con CONTAINER_NAME: D04 governa questa differenza. Non rimuovere forzatamente container omonimi di altri stack.

Dev esce con il codice del comando. Definire se errori readiness lasciano i servizi avviati; default proposto: non eliminare risorse esistenti, riportare lo stato e suggerire cleanup esplicito.

13. Immagini e volumi dei servizi

Il piano distingue render, setup, build e pull. Le azioni specifiche sono delegate agli hook dichiarati e invocati senza shell eval.

- Immagine locale personalizzata: build hook, tag risolto, verifica successiva.
- Immagine upstream: pull secondo politica approvata, versione esplicita.
- Volume external: creazione/setup prima del runtime; Compose non lo crea.
- Volume Compose-managed: lifecycle e cancellazione dichiarati.
- Non usare UID/GID host per un database senza contratto esplicito del servizio.
- Gestire più istanze e versioni incompatibili senza riutilizzare automaticamente dati.

`venv` e `uv-python` sono external, preservati da down e down -v. La pulizia di volumi esterni è un'operazione separata con conferma e nomi esatti.

14. File generati, stato e concorrenza

Proposta:

```
<project>/.docker-dev/
├── compose.user.yaml             # utente, mai sovrascritto
├── generated/
│   ├── compose.yaml              # base renderizzata
│   ├── compose.services.yaml
│   ├── profiles
│   └── plan.json
└── state/
    ├── applied.json
    └── applied-compose.yaml      # snapshot, secondo D16/D18
```

Lo snapshot applicato non è soltanto services.lock con un hash: deve consentire --down senza ripetere --with. Considerare riservatezza del modello risolto, che può contenere credenziali. Non scrivere segreti in lock/log; scegliere permessi e politica retention.

Aggiungere file user agli argomenti -f solo se esistono. Definire esplicitamente file ambiente, evitando discovery .env dipendente dalla cwd.

Lock per progetto e temporanei univoci evitano sovrascritture concorrenti. Pubblicazione atomica del modello, piano e stato coerente. Se cambia il set di servizi, segnalare servizi orfani e scegliere una politica di rimozione esplicita; non rimuoverli silenziosamente.

15. Operazioni amministrative

Operazione	Flusso e vincoli
session	Rendering, validazione, piano, conferma, setup/build, servizi, dev
--config	Mostra configurazione; nessuna build, creazione volumi o avvio
--dry-run	Mostra piano; nessun cambiamento risorse Docker; eventuali temporanei rimossi
--down	Usa stato applicato; non genera uno stack vuoto perché manca --with
--build-service	Indipendente da target; non avvia dev
clean/reset	Operazione separata, selettiva e confermata

Dispatch prima della pipeline normale. `--config` e `--dry-run` devono selezionare correttamente Compose anche senza --with. Specificare se modalità di sola configurazione richieda daemon Docker raggiungibile.

16. Compatibilità extra args e baseline

Non promettere traduzione arbitraria docker run → Compose. Mantenere risoluzione legacy; in Compose trasferire default noti e documentare le opzioni supportate. Personalizzazioni non gestite devono produrre errore o warning secondo D14, mai essere ignorate senza messaggio.

Distinguere override ambiente esplicito da fallback interno. La risoluzione iniziale prima della copia template e quella in 30.run possono differire: testare nuovi progetti e progetti esistenti.

Problemi baseline da tracciare:

ID	Osservazione	Azione proposta
B01	Setup volume ritorna se esiste, senza verifica ownership	Documentare; repair esplicito, no chown indiscriminato
B02	Volume creato ma init fallito può essere riusato	Stato inizializzazione e retry/repair definito
B03	uv-python globale condiviso tra utenti/progetti	Confermare condivisione e rischio UID differenti
B04	build passa TZ ma Dockerfile non dichiara ARG TZ	Correzione separata o inclusa esplicitamente
B05	venv non vuoto salta sync salvo UV_SYNC_ALWAYS	Preservare policy; testare e scegliere D19
B06	Stessa immagine riusata tra UID/GID differenti	Documentare/includere identità in policy immagini se necessario
B07	Parsing extra args basato su splitting whitespace	Limite legacy da non replicare nel nuovo percorso

17. Sicurezza Perl e dipendenze

Usare YAML::XS aggiornato con politica esplicita di caricamento oggetti disabilitato; non abilitare LoadBlessed. Verificare sulla versione fissata anche le opzioni di caricamento codice e i tipi booleani. Non equiparare questa misura a una sandbox completa o a validazione di schema.

Template::Toolkit con STRICT => 1, contesto completo e template fidati. Limitare percorsi e funzionalità: non abilitare valutazione Perl o plugin non necessari; parser YAML sicuro non rende sicuri hook o template arbitrari.

Dipendenze da verificare in preflight:

- interprete Perl e versione minima;
- YAML::XS e Template::Toolkit con versioni testate;
- Docker CLI/daemon nelle operazioni che lo richiedono;
- plugin Compose e feature richieste.

Installazione distro o CPAN da documentare dopo la scelta delle versioni. Esecuzione hook solo da percorsi validati del catalogo; parametri mai valutati come codice.

18. Piano di test e criteri di accettazione

Unitari

- CLI ripetibile e delimitatore --; argv con spazi e valori vuoti.
- Alias globale/progetto e servizio sconosciuto.
- Schema, tipi, chiavi sconosciute/duplicate e limiti porte.

- Project name valido e collisioni identità.
- Template mancante, STRICT e flag opzionali.
- Quoting di stringhe, liste, booleani, caratteri speciali e $ Compose.
- Aggregazione senza services ripetuti; collisioni ambiente/volumi.
- Piano senza effetti Docker; codici errore.

Integrazione

- Legacy invariato con e senza comando.
- Nuovo progetto: copia template e opzioni effettive corrette.
- Mount /app, venv, uv-python e bashrc invariati.
- Container dev one-off unico; nessun dev residente duplicato.
- Porta 8000 pubblicata con run --service-ports.
- Readiness database, timeout e servizio senza healthcheck.
- External volumes creati dai rispettivi hook e riusati.
- down senza --with usa stato precedente e conserva volumi external.
- down -v rimuove solo risorse gestite ammesse, con conferma ove esposto.
- Immagine mancante: errore/build/pull secondo politica.
- compose.user.yaml assente/presente e precedenza verificata.
- --config/--dry-run non creano volumi né avviano container.
- --build-service senza target.
- Concorrenza, init fallita, output atomico e stato non corrotto.
- Cambio set servizi e gestione orfani.
- Venv non vuoto e UV_SYNC_ALWAYS rispettano baseline.

Accettazione: decisioni bloccanti approvate, test su versione minima Compose passati, comportamento legacy preservato, nessuna logica specifica nei moduli centrali, documentazione CLI/stato/cleanup completa.

19. Materiali e chiarimenti necessari

Prima del collaudo acquisire:

- `.settings/.bashrc` e `.settings/.gitignore`, richiesti da apply-templates;
- eventuali `.docker_container` specifici Python e configurazioni progetto modificate;
- versione Compose, contesto Docker e piattaforma target;
- implementazioni reali dei primi servizi, relativi Dockerfile/entrypoint se personalizzati;
- decisioni di isolamento, naming, porte, segreti, readiness e ciclo di vita.

I file disponibili sono sufficienti per definire questa revisione, non per certificare una completa esecuzione runtime.

20. Sequenza di realizzazione proposta

1. Approvare D01–D04, D06–D10, D12–D16 e le versioni dipendenze.
2. Definire schema contesto e manifest, CLI e regole conflitti.
3. Implementare resolver/renderer Perl con test puri.
4. Integrare rendering base Python e dispatcher senza alterare legacy.
5. Implementare catalogo PostgreSQL/Redis con hook specifici.
6. Integrare readiness, stato applicato, down e locking.
7. Eseguire regressione e collaudo runtime.
8. Solo dopo stabilizzazione, pianificare migrazione main a Perl.

Conclusione: architettura approvabile come direzione; implementazione subordinata alle scelte marcate DA DECIDERE e alla correzione dei problemi C01–C12.